djbsort in Rocq

How a fast sorting routine is proved correct

Laurent Théry

INRIA, Stamp Team
Laurent.Thery@inria.fr

Abstract. djbsort is a small C library that sorts arrays of 32-bit integers. It is fast, and the comparisons it makes never depend on the values it sorts. This note explains how its two implementations, the portable one and the AVX2 one, have been proved to sort in the Rocq prover. It is written for readers who know neither sorting networks nor Rocq. Every idea comes with a small example, and the real code is shown only after the idea is clear.

1 The programs and their comparisons

First a word about the tool. Rocq is a proof assistant. It was called Coq until recently. You write definitions and statements in it, and then a proof. The machine checks every step. Nothing is accepted because it looks right. When a proof is finished, you can also ask the machine which assumptions it uses. For the proofs described here, the answer is: none.

djbsort sorts an array. It never asks "is this value smaller than that one?" and then takes one path or the other. It always runs the same small step, on a fixed pair of places in the array:

```
#define int32_MINMAX(a,b)                \
do {                                     \
  int32 ab = b ^ a;  int32 c = b - a;    \
  c ^= ab & (c ^ b); c >>= 31; c &= ab; \
  a ^= c; b ^= c;                        \
} while(0)
```

The details do not matter here. Only the effect matters. After the step, `a` holds the smaller of the two values and `b` the larger. There is no `if`. The processor runs the same instructions for every input. That is what makes the routine constant-time, and useful in cryptography.

We call one such step a **comparison**, and write it as a pair of places, for instance $(3, 5)$: compare what is in place 3 with what is in place 5, and put the smaller one in place 3.

The program never branches on the data. So the **sequence of pairs** it uses is fixed in advance. It depends only on the length of the array. A program of this shape is called a **sorting network**. The whole proof rests on this one observation.

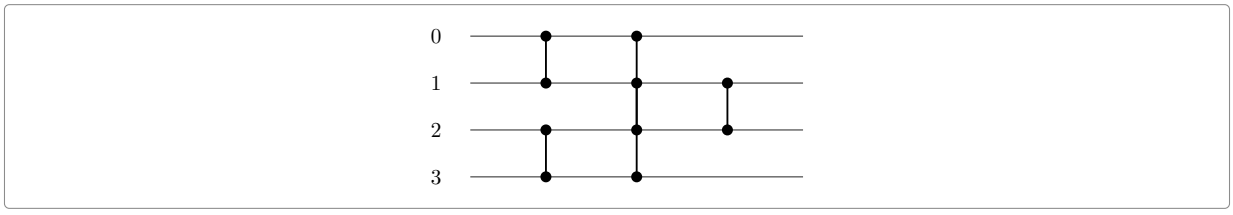

Figure 1: A network on four places. Time runs left to right. Each vertical bar is one comparison: the smaller value goes to the upper end, the larger to the lower one. This one sorts any four values.

It is worth running one input through Figure 1 by hand, because everything that follows is about such lists of pairs. The network is $(0, 1), (2, 3), (0, 2), (1, 3), (1, 2)$, and we start from $3, 1, 4, 2$:

comparison	array	what happened
–	3 1 4 2	the input
$(0, 1)$	1 3 4 2	3 and 1 were in the wrong order
$(2, 3)$	1 3 2 4	4 and 2 were in the wrong order
$(0, 2)$	1 3 2 4	1 is already the smaller
$(1, 3)$	1 3 2 4	3 is already the smaller
$(1, 2)$	1 2 3 4	the last two settle

2 Checking a network with zeros and ones

Suppose someone gives you a network on four places, like Figure 1, and says that it sorts. How can you check that? You cannot try every input: there are far too many arrays of four 32-bit integers.

There is a classical answer, and it is the first idea of the whole development.

> **The zero-one principle.** If a network sorts every array made only of zeros and ones, then it sorts every array of numbers.

Here is the reason. Take an array that the network does not sort. Look at the first place where the output is too large. Some value v ends up above a value u, with $u < v$. Now go back to the input. Replace every value below v by 0, and every value from v upwards by 1. Then run the network again. Each comparison acts exactly as before, because the smaller of two values is still the smaller after the replacement. So the zeros and ones follow the same wires as the values they replace. The output has a 1 where v was and a 0 where u was, which is again the wrong order. A network that sorts every array of zeros and ones can therefore never fail.

For four places, the check is now sixteen cases, which you can do by hand. A real array is longer. For a thousand places there are 2^{1000} cases, far too many to try, so the statement is proved instead. In fact no network in this development is checked case by case, however small it is. Section 12 explains why the machine cannot do that here, even for eight places. The machine does check some things case by case, but they are all arithmetic. For example, the tables used by the shuffles of the AVX2 code are checked at all sixty-four lanes.

3 Networks in Rocq

Before going further we must say how a network is written down in Rocq. Two ways of writing it are used, and the difference matters later.

The array. An array of m values is a `m.-tuple A`. That is exactly m values of some type `A`, and `A` carries an order. `A` can be the integers, or the booleans of the previous section. Nothing in the development depends on that choice. The places are numbered by `'I_m`, the whole numbers below m. An array of four values has places 0, 1, 2 and 3.

The first way: a list of pairs. This is what section 1 gave us, and it needs no machinery: `[:: (0,1); (2,3); (0,2); (1,3); (1,2)]` is Figure 1. The function `pnet` turns such a list into a network, and pairs whose places fall outside the array are simply dropped.

The second way: a list of stages. Hardware does not do one comparison at a time. A vector instruction does eight at once. A picture like Figure 1 is also read in columns: at each step, several pairs are compared together, and no two of them share a place. One such column is called a **connector**. It is the type the mathematical side is built from:

```
Record connector (m : nat) := connector_of {
  clink  : {ffun 'I_m -> 'I_m};
  cflip  : {ffun 'I_m -> bool};
  cfinv  : [forall i, clink (clink i) == i];
  cflipinv : [forall i, cflip (clink i) == cflip i] }.
```

Word by word:

- `clink` says, for each place, which place it is paired with. A place that is paired with itself is left alone by this stage.
- `cflip` says, for each place, which way round its pair goes: `false` for "smaller value to the smaller place", `true` for the other way.
- `cfinv` is a condition, not data. Following `clink` twice brings you back to where you started. This says that the pairing really is a set of pairs, and that no two of them share a place.
- `cflipinv` is the other condition. The two ends of a pair agree on the direction. It would make no sense for one end to want the smaller value and the other end the larger.

Running one stage is then what you would expect: each place looks at its partner and keeps the min or the max.

```
Definition cfun c t :=
  [tuple let min := min (tnth t i) (tnth t (clink c i)) in
         let max := max (tnth t i) (tnth t (clink c i)) in
         if i <= clink c i
         then if cflip c i then max else min
         else if cflip c i then min else max | i < m].
```

Read it in this way. For every place `i`, take the two values `t i` and `t (clink c i)`. If `i` is the smaller of the two places, keep the min, unless the flip says otherwise. If `i` is the larger, keep the max, unless the flip says otherwise. Both ends of a pair use the same line of code, and they agree because of `cflipinv`.

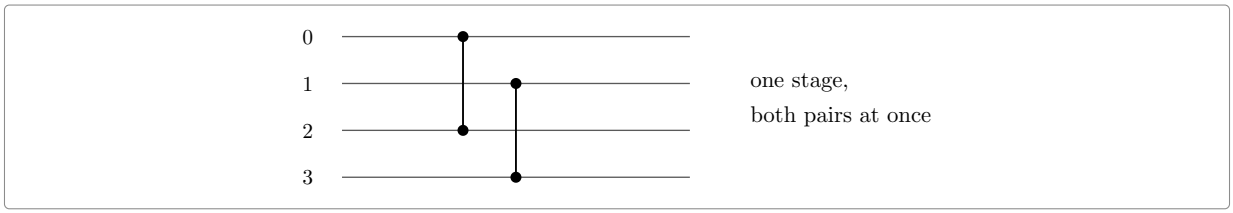

Figure 2: One connector on four places: `clink` pairs 0 with 2 and 1 with 3, and `cflip` is false everywhere. The two comparisons touch four different places, so the machine may do them together.

A **network** is then a list of stages, and running it means running them one after the other:

```
Definition network := seq (connector m).
Definition nfun n t := foldl (fun t c => cfun c t) t n.
```

The two ways of writing a network fit together. A single pair (i, j) is a stage that touches only two places (`cswap i j`). So a list of pairs is a network in which every stage does one comparison. That is exactly what `pnet` builds. The program side of the proof uses lists of pairs, because that is what a program performs. The mathematical side uses stages, because that is what induction works on.

Now the property we care about can be written down. It is the zero-one principle of the previous section, taken as the definition:

```
Definition sorting :=
  [qualify n | [forall r : m.-tuple bool, sorted <=%O (nfun n r)]].
```

- `m.-tuple bool` is an array of exactly `m` booleans, so `r` ranges over arrays of zeros and ones;
- `nfun n r` runs the network `n` on the array `r`;
- `sorted <=%O` says the result is in increasing order;
- `[forall r ...]` says this holds for every such array;
- `n \is sorting` is then read "n is a sorting network".

By the zero-one principle, this single line is as strong as sorting arrays of integers. The library proves that once, for any network.

3.1 Why the mathematical networks sort

The AVX2 code follows Batcher's **bitonic** sorter. It is worth seeing why that works, because the whole right-hand side of the proof rests on it.

A sequence is **bitonic** if it goes up and then down, like 1, 4, 7, 6, 2. A sequence that becomes such a sequence after a rotation is bitonic as well. Two sorted runs, one going up and one going down, always give a bitonic sequence when you put them end to end.

Now take a bitonic sequence of $2m$ values. Compare each place i with the place $i + m$, and keep the smaller one on the left. This single stage is the **half-cleaner**. After it, three things are true. They are the key to everything that follows:

1. every value in the left half is at most every value in the right half;
2. the left half is still bitonic;
3. the right half is still bitonic.

So the problem splits in two. No value has to cross the middle again, and each half is a smaller copy of the same problem. Repeat the half-cleaner on the halves, then on the quarters, and so on, and the whole array is sorted. That is `half_cleaner_rec`.

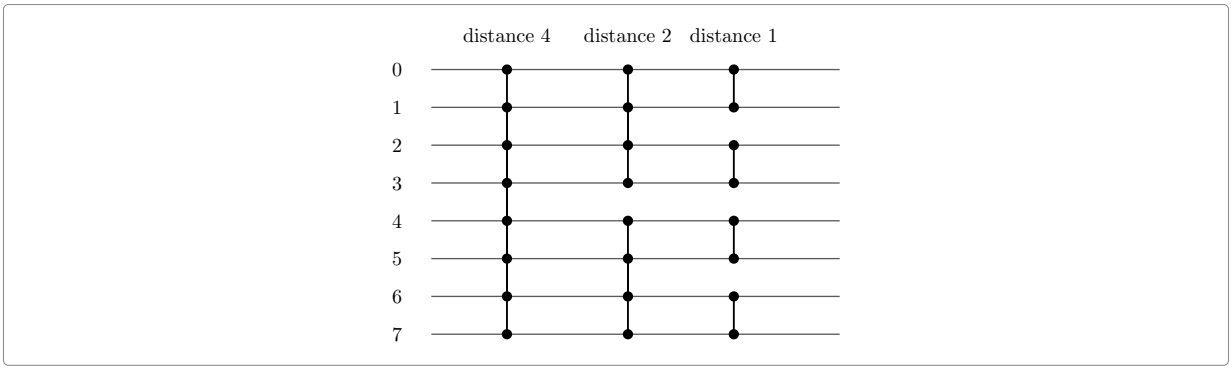

Figure 3: Sorting a bitonic sequence of eight values: one half-cleaner at distance four, then one in each half at distance two, then one in each quarter. After the first column, nothing crosses the middle again.

A full sorter follows. Sort the first half upwards, sort the second half downwards, and put them together. The result is bitonic, so Figure 3 finishes the work. That is a recursion, and it is the network the AVX2 code uses.

The portable code follows a different plan, Knuth's merge exchange. The shape of the argument is the same: a network built by a recursion, proved to sort by induction, with the zero-one principle doing the work at the bottom.

4 The shape of the proof

There are two objects, and they are not of the same kind.

On one side there is a **network**. Mathematics knows how to reason about it, and it can be proved to sort by induction. For the AVX2 code that network is Batcher's bitonic sorter, and for the portable code it is Knuth's merge exchange. On the other side there is a **program**, made of loops, vector registers, shuffles and masks.

The proof therefore has three parts: the network, the program, and a bridge. The bridge says that the program performs the comparisons of the network.

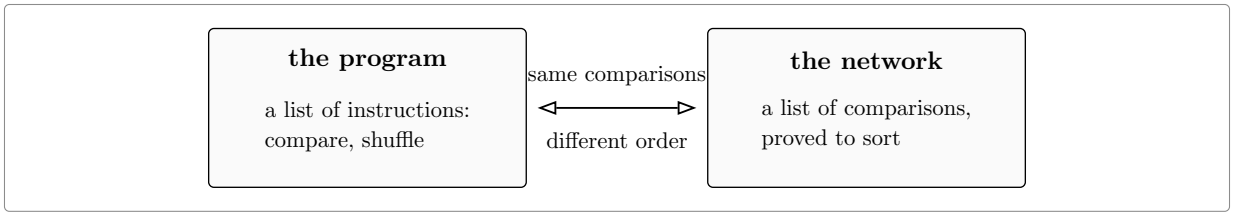

Figure 4: The two sides and the bridge between them. The left box is what the machine runs. The right box is what we can reason about. The bridge says that they perform the same comparisons.

The left box is built once and for all, as a small language. The right box is classical mathematics. Almost all the work is in the bridge. The rest of this note describes the techniques that the bridge needs.

5 Technique one: writing the program down

A program is written as a list of instructions. There are only three of them:

```
Inductive item : Type :=
  | Cmp   of nat * nat            (* one compare-exchange *)
  | Vcmp  of seq (nat * nat)      (* one vector compare-exchange: its lanes *)
  | Vshuf of cperm m.             (* one vector lane shuffle *)
```

- `Cmp (3, 5)` is the scalar step of section 1: compare places 3 and 5.
- `Vcmp` is the same thing done by the vector unit. One AVX2 instruction compares eight pairs at once, so the instruction carries the list of the eight pairs it performs.
- `Vshuf` moves values about without comparing anything. This is what a lane shuffle or a transpose does.

A list of such instructions is a `prog`. There are two ways to read a program, and the difference between them is the main point of this section.

Running it. `pfun p t` takes an array `t` and returns the array after the program has run. This is what the machine does.

Reading it. `pflat p` walks through the program and collects the pairs it compares. It ignores the shuffles. The result is a plain list of comparisons, which is the right-hand box of Figure 4.

There is a difficulty: a shuffle changes what "place 3" means. Suppose a program first exchanges the values in places 1 and 2, and then compares places 0 and 1. In terms of the values, that comparison is between the value that started in place 0 and the value that started in place 2.

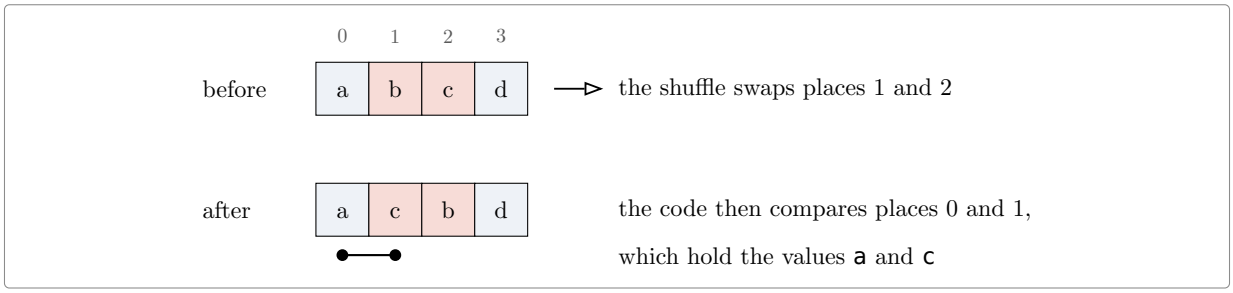

Figure 5: A shuffle changes the meaning of a place. The comparison that the code writes as $(0, 1)$ is, in terms of values, the pair (a, c).

So `pflat` does not record the pair that the code writes down. It records the pair **renamed by all the moves made so far**. In the example it records $(0, 2)$, because the value called `c` started in place 2. Every comparison is then named in one fixed way, and the list can be compared with a network.

One more point explains how the AVX2 proof is organised. The program ends with a large shuffle, because the values come out of the vector registers in an order that must be undone. Following that at every step would be painful. Instead, every comparison is named by **the place its values end up in** when the program finishes. Nothing is lost. It is a change of names, and it is applied everywhere.

5.1 What the renaming gives

That change of names is not only useful for the bridge. It is what makes the AVX2 code readable at all. Read as it is written, `code/avx2/c/sort.c` is a reversing pass, then a ladder of stages of growing width, with masks that flip signs along the way. Nothing in it looks like a network for which we have a theorem. A trace of the code does not help either. Run it on an array whose entries are their own place numbers, and record what it compares: the comparisons look random, because the shuffles have already moved the values.

Now name each comparison by the place its two values **end up** in. The schedule becomes clear. The array is cut into groups of four. Each group is sorted, and two neighbouring groups are sorted in opposite directions: the first goes down, the second goes up. That is what a merge needs. The groups are then merged, at eight, at sixteen, at thirty-two, at sixty-four. Each merge is the usual sequence of halving distances. This is Batcher's bitonic sorter, with two differences.

The base. A group of four is sorted in five comparisons, at distances 1, 1, 2, 2 and 1. A bitonic sorter of four would use six. That one comparison saved per group is the whole size difference against the textbook network: 656 comparisons instead of 672, at sixty-four elements.

The direction. In every merge but the last, the two halves go the opposite way round from the textbook rule: the lower half goes down and the upper half goes up. Only the last merge sorts the whole array upwards. So a comparison with textbook bitonic fails, and it fails everywhere, until the rule is turned round.

Neither difference costs anything in the mathematics. The network is written down as the code has it,

```
Definition net4 (b : bool) : network (`2^ 2) :=
  pnet _ (if b then [:: (1,0); (3,2); (2,0); (3,1); (2,1)]
          else [:: (0,1); (2,3); (0,2); (1,3); (1,2)]).

Fixpoint dsort (b : bool) k : network (`2^ k.+2) :=
  if k is k1.+1
  then nmerge (dsort true k1) (dsort false k1) ++ half_cleaner_rec b k1.+3
  else net4 b.
```

and it sorts (`sorting_dsort`). The proof is the induction of section 3 with the two halves the other way round. An array that goes down and then up is bitonic as well, so the argument is unchanged. The base case has four wires, and it is settled by running through the sixteen arrays of zeros and ones.

The renaming itself is free. Push every move back past the comparison before it, again and again. A program that compares and moves then becomes **one** rearrangement of the array, followed by a network. In that network, the comparisons are named by the place each value ends in. The rearrangement now applies to the input, where it does no harm: a network that sorts does so whatever order it is given.

```
Corollary sorted_pfun (p : prog) t :
  pnetwork p \is sorting -> sorted <=%O (pfun p t).
```

6 Technique two: the same comparisons in a different order

Now both sides are lists of comparisons. We would like them to be equal. They are not, and they cannot be.

The network works **distance by distance**. It compares everything a thousand places apart, then everything five hundred places apart, and so on. The program cannot do that. A vector instruction handles eight pairs at once, and it is only worth using if all eight pairs are ready. So the code works **region by region**. It takes a block of the array, does every distance inside that block while the values are in registers, and only then moves on.

Both orders contain the same comparisons. The question is whether the order matters.

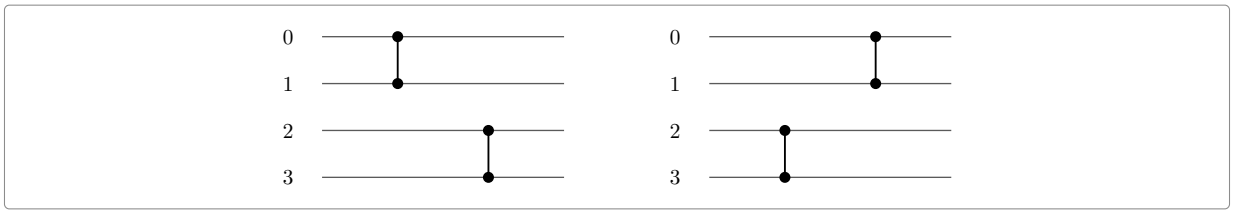

Figure 6: The same two comparisons in the two possible orders. They touch four different places, so nothing can change: whichever is done first, the result is the same.

That is the whole idea. Here it is in one sentence.

> **Two comparisons that share no place may be swapped.** Doing $(0, 1)$ then $(2, 3)$ gives the same array as doing $(2, 3)$ then $(0, 1)$.

In Rocq, "sharing no place" is a small test on two pairs. One swap is one step of a relation between lists:

```
Definition dpair (ab cd : nat * nat) : bool :=
  [&& ab.1 != cd.1, ab.1 != cd.2, ab.2 != cd.1 & ab.2 != cd.2].

Inductive dswap (n : nat) : seq (nat * nat) -> seq (nat * nat) -> Prop :=
  dswap_step ps ab cd qs of
    bnd n ab & bnd n cd & dpair ab cd :
      dswap n (ps ++ ab :: cd :: qs) (ps ++ cd :: ab :: qs).

Inductive dequiv (n : nat) : seq (nat * nat) -> seq (nat * nat) -> Prop :=
| dequiv_refl l : dequiv n l l
| dequiv_step l1 l2 l3 of dswap n l1 l2 & dequiv n l2 l3 : dequiv n l1 l3.
```

- `dpair ab cd` says the two pairs have no place in common.
- `dswap n l1 l2` says that the two lists are the same, except at one point: two neighbouring comparisons that share no place have changed order. `bnd n` only says that the places are inside the array.
- `dequiv n l1 l2` says: one list can be turned into the other by a run of such swaps.

And the payoff, proved once:

```
Lemma nfun_dequiv n (l1 l2 : seq (nat * nat)) (t : n.-tuple A) :
  dequiv n l1 l2 -> nfun (pnet n l1) t = nfun (pnet n l2) t.
```

In words: two lists related by `dequiv` compute the same function. So if the program's list is a reordering of the network's list, and the network sorts, then the program sorts as well.

6.1 Justifying a whole rearrangement at once

Swapping neighbours is fine on paper, but the two orders differ by millions of swaps. We cannot write them all out. We need a way to justify a whole rearrangement at once. Here it is.

Give every comparison a **colour**, with two conditions:

1. two comparisons of different colours never share a place;
2. both lists, read from left to right, contain each colour in the same order.

Then the two lists are related by `dequiv`. The reason is short. A comparison only has to move past comparisons of another colour, and by the rule above it may move past those freely.

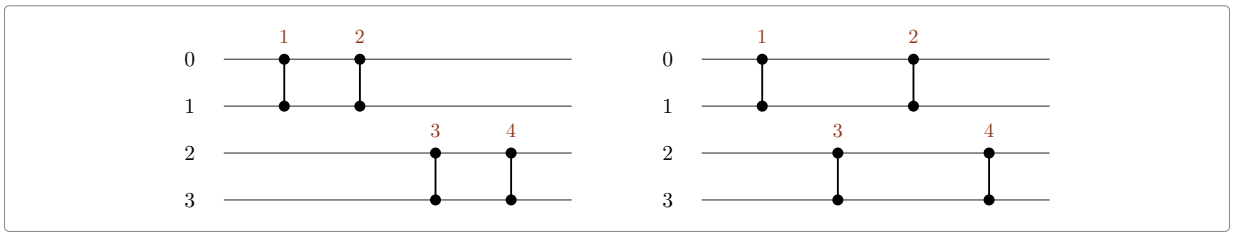

Figure 7: The colour of a comparison is the half of the array where it sits. The left list does the top half first. The right list takes the two halves alternately. Different colours never share a place, and each colour appears in the same order in both lists (1 then 2, and 3 then 4). So the two lists compute the same thing.

This is exactly the situation of Figure 4. The program goes region by region, and the network goes distance by distance. The colour of a comparison is the region it belongs to. In Rocq the statement is

```
Lemma dequiv_colour n (c : nat * nat -> nat) (m : nat) (l1 l2 : seq (nat * nat)) :
  ... (* both lists are in range, and every colour is below m *)
  (forall g, g < m -> [seq ab <- l1 | c ab == g] = [seq ab <- l2 | c ab == g]) ->
  dequiv n l1 l2.
```

Here `c` is the colouring, and the last line is condition 2: keeping only one colour gives the same list on both sides. This single lemma settles the reordering in both programs. Only the colouring changes. For the portable code it is the place where a chain of comparisons starts. For the AVX2 code it is the group of eight, or of sixty-four, that a value belongs to.

7 Technique three: sorting downwards without any downward code

Networks like the bitonic sorter do not only sort upwards. Half of the work is sorting a block **downwards**, so that the two halves together can be merged.

The code contains no downward comparison. It uses a trick instead. Before a block is sorted downwards, every value in it is complemented. All its bits are flipped, which reverses the order. The block is then sorted upwards as usual, and complemented back at the end. Complementing twice does nothing, so the complements of neighbouring stages cancel each other. In the real code the flips are kept in a running **mask**. The mask says, for each place, whether the value sitting there is complemented at that moment.

Here is an example with two values. Sorting $(3, 7)$ downwards should give $(7, 3)$. Complement the two values: 3 becomes -4 and 7 becomes -8, so the pair is $(-4, -8)$. Sort it upwards: $(-8, -4)$. Complement back: $(7, 3)$, which is what we wanted. The sort itself never knew about the direction.

The proof therefore has to carry that pattern. It is written as a predicate:

```
Definition dflP (K : nat) (fl : flips) : Prop :=
  size fl = n /\ forall i, i < n -> nth false fl i = dfl K i.
```

It reads: the mask `fl` has one bit per place, and its bit at place `i` is the bit that the merge of size `K` asks for. The proof then shows that every pass of the program keeps this property. A shuffle carries the mask with the data, and a mask instruction adds a pattern into it, bit by bit. After the last merge the mask is false everywhere, so nothing is complemented. That is what “sorted upwards” requires.

8 Technique four: loops instead of a recursion

Textbooks write the bitonic sorter as a recursion: sort the two halves in opposite directions, then merge. The code does not do that. It runs three loops inside one another, and the innermost line is the well known one:

```
for (k = 2; k <= n; k *= 2)
  for (j = k/2; j >= 1; j /= 2)
    for (i = 0; i < n; i++)
      if ((i & j) == 0)
        compare(i, i ^ j, ascending: (i & k) == 0);
```

Two whole numbers decide everything. The partner of place `i` is `i` with one bit turned over, and the direction is another bit of `i`. Nothing else is involved.

Are the three loops and the recursion the same network? Yes, and this is now a theorem, not a belief. Take eight places. The loops run, in this order:

round	distances	what it achieves	in the recursion
k = 2	1	pairs sorted, alternately up and down	the two halves of a half
k = 4	2, 1	runs of four, alternately up and down	the two halves
k = 8	4, 2, 1	the whole array sorted	the final merge

In the first two rounds the two halves of the array are sorted in opposite directions, which is what the recursion asks for. The last round is the merge. The bit `i & k` is what makes the second half go the other way. In Rocq this is proved as an equality of networks, stage by stage, including the flips:

```
Theorem isort_pbsort k : isort k = pbsort false k.
```

`isort` is the loop version, written down as a network. `pbsort` is the recursive sorter, proved to sort by induction. Before this theorem, the two were only compared by running them on small sizes.

9 Technique five: reading the loops of the portable code

The portable code is Knuth's merge exchange. It raises the same difficulty in another form. Its inner loop follows a **chain**. It takes one place and compares it with places further and further away, halving the distance each time. The network groups the comparisons the other way: all the longest ones first, then all the next longest, and so on.

So once again the two lists hold the same comparisons in a different order, and the same colouring method applies. Two facts do all the work:

- two ways of running a pass differ only by comparisons on even places against comparisons on odd places, and those never share a place.
- the longest comparison may be taken out of every chain, because the long comparison of a later chain shares no place with the short comparisons of an earlier one.

Both are special cases of the lemmas of section 6. Both were once proved from scratch, at the level of functions rather than lists. The two programs are very different, but the argument is the same one twice.

10 Technique six: a length that is not a power of two

A network is built for a fixed number of wires, and every sorter in this note has a power of two of them. Real code must sort eleven elements as well. There are two solutions, and djbsort uses both.

The first one is **padding**. Fill the end of the array with a value above everything else, sort the whole array, and drop that filling. The sort has pushed it to the end, so what remains at the front is the answer. This is what the C does for a short array, and it is what `avx2_prog_pad` says.

The second one is **splitting at the top bit**. This is what `int32_sort_short` does for a long array. Eleven is $8 + 2 + 1$. Sort the first eight downwards, then sort the remaining three in the same way, by calling the routine itself. The array now goes down and then up. That is exactly the shape a bitonic merge can handle, so one merge finishes the work. Remove the top bit again and again: the blocks of the recursion are simply the bits of n, from the highest one down.

But a merge of eleven wires is not a network either. The code takes the merge of the next power of two, sixteen, and drops every comparison that reaches past place ten. It is worth explaining why this is correct. It is the one place where padding comes back as an argument instead of as code. Imagine the five missing places filled with a value above everything else. A comparison between two real places behaves as before. A comparison that reaches into the imaginary part compares a real value with that top value. The smaller one is the real value, and it stays at the low place. The top value goes to the high place, which is imaginary. So nothing in the array moves. The pruned merge therefore does to the eleven places what the full merge would do to the padded sixteen, and the full merge sorts. That is `nfun_pnet_padt`, and the statement it gives is

```
Theorem sorted_hcr_prune (p n : nat) (s1 s2 : seq bool) (t : n.-tuple bool) :
  n <= `2^ p -> sorted >=%O s1 -> sorted <=%O s2 -> t = s1 ++ s2 :> seq bool ->
  sorted <=%O
    (nfun (pnet n [seq ab <- nstages (half_cleaner_rec false p) | ab.2 < n]) t).
```

It reads: an array that goes down and then up is sorted by the merge of the next power of two, once the comparisons that leave the array are removed.

There is a trap in that argument. It only works when a comparison sends the smaller value to the **lower** place. Every comparison of the merge does that. The comparisons of the block sorter do not: half of them run downwards. This is the mask trick of section 7, seen on the list side. So the two solutions cannot be exchanged. The merge may be pruned, but the blocks must be padded.

At the level of lists, sorting a block downwards is one line: turn every comparison round. The upward sorter compares (a, b) and puts the smaller value in a. The downward one compares (b, a). The proof that this really sorts downwards is the complement argument of section 7 again, in two lines. Flipping every value turns one comparison round, and flipping an increasing list gives a decreasing one.

Put together, one recursive call is

```
Theorem sorting_smerge (p q m : nat) (l1 l2 : seq (nat * nat)) :
  q + m <= `2^ p ->
  all (fun ab => (ab.1 < q) && (ab.2 < q)) l1 ->
  pnet q l1 \is sorting -> pnet m l2 \is sorting ->
  pnet (q + m) (smerge p q m l1 l2) \is sorting.
```

It says: take anything that sorts the first block, and anything that sorts the rest; with the merge on top, the whole array is sorted. The recursion is this lemma applied to the bits of n. Use djbsort's own AVX2 comparisons as the block sorter, and you get `sorting_avx2_short`, in `code/avx2/proof/sort_short.v`. That is a sorting network for every length.

One more change of view is needed before the code can be compared with this. The merge is **written** as a recursion: one cleaner across the whole array, then two copies of itself on the two halves. No code runs it that way. Code runs one distance at a time, across the whole array: every comparison at distance $\frac{n}{2}$, then every one at $\frac{n}{4}$, down to 1. The two descriptions give the same list, and `code/common/nlevel.v` proves it:

```
Theorem nstagesl_half_cleaner_rec (p : nat) :
  [seq cpairs c | c <- half_cleaner_rec false p]
    = [seq level_pairs (`2^ p) d d false | d <- dists p].
```

Here `level_pairs N d d false` is one level. It holds every pair $(i, i + d)$ with $i + d < N$ and with the d-bit of i equal to zero, listed by increasing i. Two small facts carry the induction. One cleaner

is one level. And putting two copies of an array side by side doubles each level exactly. The second fact needs $2d$ to divide the half-width, which is true here, because every distance is a power of two. Pruning only shortens each level, so the merge on a length that is not a power of two is `flatten [seq level_pairs n d d false | d <- dists p]`.

That list is what the loops of `code/avx2/c/sort_short.c` must be compared with. The first half of the comparison is easy. Look at what the code does at one distance:

```
long long j = stage(x,n,8,q,N_mrg8);
minmax_vector(&x[j], &x[j + 4*q], n - 4*q - j);
```

`stage` runs the whole blocks while they fit, and returns the place where it stopped. `minmax_vector` then handles the piece that is left over, with a length computed from n. A level has exactly that shape. Moreover the two sides are equal as lists, and not only as sets, because both list the comparisons by increasing low place:

```
Theorem level_pairs_blocks (n d : nat) : 0 < d ->
  level_pairs n d d false
    = flatten [seq mm (m * d.*2) (m * d.*2 + d) d | m <- iota 0 (n %/ d.*2)]
        ++ mm (n %/ d.*2 * d.*2) (n %/ d.*2 * d.*2 + d)
              (n - d - n %/ d.*2 * d.*2).
```

Here `mm a b len` is what `minmax_vector(&x[a], &x[b], len)` compares. Take eleven wires at distance two. They give two whole blocks and then an end piece of one comparison: $(0, 2), (1, 3)$, then $(4, 6), (5, 7)$, then $(8, 10)$. Subtraction on whole numbers does real work here. When fewer than d places are left, the length of the end piece comes out as zero. That is exactly what the C does: `n - 4*q - j` becomes negative, and `minmax_vector` returns without comparing anything.

The second half of the comparison is the subject of the next section. It is what the code does **inside** a block, where it works on eight lanes at a time and does three distances in one pass.

11 Technique seven: eight lanes at a time

A level at distance d compares place i with place $i + d$. The code never goes through it in that way.

One distance, eight at a time. The code loads eight consecutive places into one register, and the eight places q further on into another. It compares the two registers: one instruction, eight comparisons. Then it moves on by eight lanes. The groups of eight come in increasing order, so this is not even a reordering. The list the code produces is the level itself, comparison for comparison:

```
Theorem level_pairs_vstage (n q : nat) : 0 < q -> 8 %| q ->
  level_pairs n q q false
    = vstage n 2 q [:: (0, 1)]
      ++ mm (n %/ q.*2 * q.*2) (n %/ q.*2 * q.*2 + q)
            (n - q - n %/ q.*2 * q.*2).
```

Three distances at once. Once eight registers are loaded, the code does not stop at one distance. `N_mrg8` is a table of twelve compare-exchanges **between registers**. Running it does the work of three levels, while the values stay where they are. That is where the reordering appears.

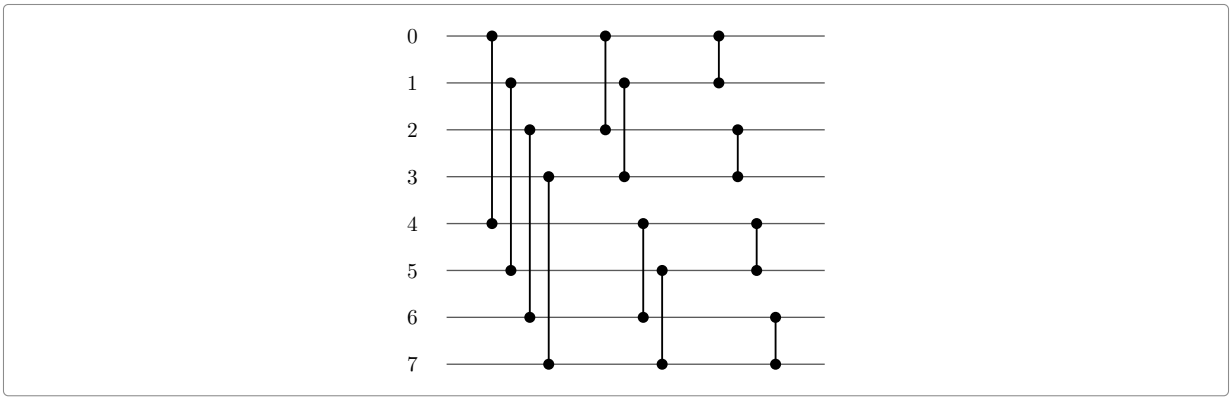

Figure 8: The table `N_mrg8`: twelve comparisons between eight registers. It is the bitonic merge on eight wires, one distance at a time. Each wire here is q consecutive places of the array, so each comparison drawn is q comparisons of the array, and the three columns are the levels at $4q$, at $2q$ and at q.

Two observations turn that picture into a proof.

The first one: the tables **are** merges. The merge on eight wires, read one distance at a time, is `P_mrg8` of the C source, letter for letter. The same holds for four registers, and for two:

```
Example nstages_hcr_3 :
  nstages (half_cleaner_rec false 3)
    = [:: (0, 4); (1, 5); (2, 6); (3, 7); (0, 2); (1, 3); (4, 6); (5, 7);
          (0, 1); (2, 3); (4, 5); (6, 7)].
Proof. by rewrite nstages_half_cleaner_rec. Qed.
```

The second one: a register stands for q consecutive places. Replace every wire of a level by q places, and you get a level again, at q times the distance. This holds comparison by comparison, and inside each comparison, lane by lane:

```
Theorem level_pairs_scale (N d q : nat) : 0 < d -> 0 < q ->
  level_pairs (N * q) (d * q) (d * q) false
    = flatten [seq mm (ab.1 * q) (ab.2 * q) q | ab <- level_pairs N d d false].
```

Together they say that the table, run on a block of $8q$ places, gives the levels at $4q$, $2q$ and q of that block. Only the order inside the block is left to justify. The code goes group of eight by group of eight, while the levels go comparison by comparison. Colour a comparison by its group of eight. For a place x, that colour is `x %% q %/ 8`. Section 6 then settles the question, for **any** table of comparisons between registers:

```
Theorem vblock_dequiv (n base q c : nat) (g : seq (nat * nat)) :
  0 < q -> 8 %| q -> q %| base -> base + c * q <= n ->
  all (fun ab => (ab.1 < c) && (ab.2 < c)) g ->
  dequiv n (vblock base q q g)
           (flatten [seq mm (base + ab.1 * q) (base + ab.2 * q) q | ab <- g]).
```

One turn of the loop. Now the loop itself can be read. It does three distances at a time:

```
while (q >= 64) {
  q >>= 2;
  long long j = stage(x,n,8,q,N_mrg8);
  minmax_vector(&x[j], &x[j + 4*q], n - 4*q - j);
  if (j + 4*q <= n) { blockn(x,j,q,q,4,N_mrg4); j += 4*q; }
  minmax_vector(&x[j], &x[j + 2*q], n - 2*q - j);
  if (j + 2*q <= n) { blockn(x,j,q,q,2,N_mrg2); j += 2*q; }
  minmax_vector(&x[j], &x[j + q], n - q - j);
  q >>= 1;
}
```

The stage runs the whole blocks of $8q$ places, and each block does the three levels at once. The six lines after the stage pick up what it could not reach. The level at $2q$ may have one whole block more than the stage ran, and the level at q up to three more. All three levels also have an end piece. The counts come from what n leaves over $8q$. Write r for that remainder. The level at $2q$ has $2(n \div 8q) + r \div 4q$ whole blocks, and the level at q has $4(n \div 8q) + r \div 2q$. The extra blocks that the code runs are exactly the missing ones.

Then comes the order, and here the picture is simple. Everything the stage does lies below the place j where it stopped. Everything after it starts at j or later. Two comparisons on opposite sides of j share no place. So three moves of whole blocks put the turn in level order: first the whole level at $4q$, then the whole level at $2q$, then the whole level at q.

One detail does not fit this picture. When the length given to `minmax_vector` is not a multiple of eight, the routine does the **last** eight places first, and then walks through the whole eights from the start. So eight of its comparisons are done twice. A reordering cannot explain that: two lists that differ by a repeated comparison are not rearrangements of one another. But they do compute the same function, because doing a comparison twice is the same as doing it once:

```
Definition nequiv (n : nat) (l1 l2 : seq (nat * nat)) : Prop :=
  forall (d : disp_t) (A : orderType d) (t : n.-tuple A),
    nfun (pnet n l1) t = nfun (pnet n l2) t.

Lemma nequiv_dup (n a b : nat) (l : seq (nat * nat)) : a < n -> b < n ->
  nequiv n ((a, b) :: (a, b) :: l) ((a, b) :: l).
```

Every reordering is an `nequiv`, and so is the removal of a repeated comparison. With these two facts, one turn of the loop is three levels of the merge:

```
Theorem mbody_levels (n q : nat) : 0 < q -> 8 %| q ->
  nequiv n (mbody n q)
           (level_pairs n (4 * q) (4 * q) false
            ++ level_pairs n (2 * q) (2 * q) false
            ++ level_pairs n q q false).
```

Here `mbody n q` is the turn written out: the stage, the two extra blocks and the three end pieces, in the order in which the C runs them.

11.1 All the turns of the loop

One turn is three levels, and the loop is all of them. Look at what the loop does to q. It divides q by four before the body and by two after it, so each turn divides q by eight. Three halvings, three levels. Nothing else about the loop matters. Write 2^j for the value of q in the body on the last turn. Then k turns are

```
Fixpoint mturns (n j k : nat) : seq (nat * nat) :=
  if k is k1.+1 then mbody n (`2^ (j + 3 * k1)) ++ mturns n j k1 else [::].
```

One line of `mbody_levels` per turn gives the $3k$ levels that they run. These levels are `dtop j k`, largest distance first. The loop stops before q falls below eight lanes, so $j \geq 3$. That is the only condition.

One list identity makes this the **whole** merge. Those $3k$ distances are the largest ones of a merge on 2^{j+3k} wires, and what remains below them is a merge on 2^j wires.

```
Lemma dists_dtop (j k : nat) : dists (j + 3 * k) = dtop j k ++ dists j.
```

So take the loop, and after it anything that runs the distances at which the loop stopped. Together they are the pruned merge of section 10, and therefore they sort:

```
Theorem sorted_mturns (n j k : nat) (L : seq (nat * nat)) (s1 s2 : seq bool)
    (t : n.-tuple bool) :
  3 <= j -> n <= `2^ (j + 3 * k) ->
```

```
  nequiv n L (flatten [seq level_pairs n d d false | d <- dists j]) ->
  sorted >=%O s1 -> sorted <=%O s2 -> t = s1 ++ s2 :> seq bool ->
  sorted <=%O (nfun (pnet n (mturns n j k ++ L)) t).
```

The hypothesis on `L` is a contract with the rest of the code. The last phase of `int32_sort_short` may do what it likes. If it performs the levels at 2^{j-1} down to 1, then the merge is complete.

An example with a thousand elements. The code enters the merge with $q = 512$. The first turn runs the levels at 512, 256 and 128, and the second one those at 64, 32 and 16. Then q is 8, the test `q >= 64` fails, and the loop is over. Here $j = 4$ and $k = 2$: two turns and six levels. The merge on $2^{4+6} = 1024$ wires is the one that covers a thousand places. The contract left for the last phase is the four levels at 8, 4, 2 and 1.

12 Technique eight: comparing against a trace

At the small end of the size range the code is no longer a loop. For eight, sixteen and thirty-two elements it runs a fixed sequence: a table of comparisons for eight, and for the other two sizes a few vector instructions with no loop at all. There is nothing to reason about here. There is only one thing to **check**: are those comparisons a sorting network?

The obvious answer is to ask the machine, since there are only sixteen wires. That does not work, and the reason decides how the rest is done.

In this development a network is a list of connectors. A connector is a finite function on the places of the array, together with two proofs (`clink` and `cflip` above). These proofs make a connector a sound object, but they also stop it from computing. Ask Rocq to evaluate the comparisons of the bitonic sorter on eight wires, and you get a term the size of a small book, blocked on the proof that 3 is below 4. Deciding "this network sorts" is even worse. The zero-one principle turns the question into 2^8 runs, and each run carries the same blocked machinery, so the computation never ends.

A list of pairs of ordinary numbers does compute. So the small sorters are written a second time, as a list:

```
Definition mlev (k : nat) : seq (nat * nat) :=
  flatten [seq level_pairs (`2^ k) d d false | d <- dists k].

Fixpoint bsl (k : nat) (up : bool) : seq (nat * nat) :=
  if k is k1.+1
  then (bsl k1 false ++ pshift (`2^ k1) (bsl k1 true))
       ++ (if up then mlev k1.+1 else rpairs (mlev k1.+1))
  else [::].
```

`bsl k up` is the bitonic sorter on 2^k wires. It sorts the first half downwards and the second half upwards, and then merges. When the whole block is wanted downwards, the last merge is turned round, which is exactly what the masks of the code do. It sorts in the direction asked for:

```
Theorem sorted_bsl (k : nat) (up : bool) (t : (`2^ k).-tuple bool) :
  sorted (if up then (<=%O : rel bool) else >=%O)
         (nfun (pnet (`2^ k) (bsl k up)) t).
```

The proof is two lines of the induction used in section 10. The two halves leave the array going down and then up, which is bitonic, and the merge finishes the work.

And now `bsl` can be **printed**. At $k = 3$ it has twenty-four comparisons. Put them next to the trace of the C at sixteen elements. That trace comes from the OCaml copy of the code in `code/avx2/ml/trace_short.ml`, which runs the same instructions on place numbers instead of values. The two lists hold the same comparisons in a different order. The code sorts the two halves in two registers at the same time, so it alternates between them, while `bsl` does one half and then the other.

Two comparisons in different halves share no place. So this difference is again the reordering of section 6, and one lemma gives it:

```
Lemma dequiv_cut (n b : nat) (l : seq (nat * nat)) :
  all (bnd n) l -> all (fun ab => (ab.1 < b) == (ab.2 < b)) l ->
  dequiv n l ([seq ab <- l | ab.1 < b] ++ [seq ab <- l | ~~ (ab.1 < b)]).
```

Take a list in which no comparison crosses a given line. It may be read as everything below the line, then everything above it. The colour of a comparison is the side of the line where it sits.

With this lemma, the trace at sixteen becomes the sorter in three cuts: at 8, then at 4 and at 12. The trace at thirty-two needs four cuts. Three of them are the same, inside its first block, and one is inside its merge, because the code merges each half as a whole while the levels take the two halves alternately. Every step in between is an identity between lists, which the machine settles by evaluation. That is what `bsl` was written for. The results are `dequiv_c16` and `dequiv_c32`, and hence `sorting_c16` and `sorting_c32`. The blocks of sixteen and thirty-two wires in the recursion of section 10 are now sorted by the code's own comparisons. Below sixteen the C has no fixed sequence at all. It sorts eight elements or fewer with a bubble sort, so at those sizes a sorter of the same width is still used in its place.

13 Technique nine: the phase that finishes the merge

The loop of section 11 stops when q falls below sixty-four. The levels at 2^{j-1} down to 1 are then still to do. They are the contract `L` of `sorted_mturns`. The piece of code that follows the loop does them, and it never walks through the whole array:

```
long long j = 0;
for (int w = (int)(q >> 2); w >= 2; w >>= 1) {
  net_t first = (w == 8) ? N_mrg8 : (w == 4) ? N_mrg4 : N_mrg2;
  while (j + 8*w <= n) { bmerge(x,j,w,first); j += 8*w; }
  minmax_vector(&x[j],&x[j + 4*w], n - 4*w - j);
}
if (j + 8 <= n) { snet(&x[j],P_tail8,12); j += 8; }
minmax_vector(&x[j],&x[j+4], n - 4 - j);
if (j + 4 <= n) { snet(&x[j],P_mrg4,4); j += 4; }
if (j + 3 <= n) s_minmax(&x[j],&x[j+2]);
if (j + 2 <= n) s_minmax(&x[j],&x[j+1]);
```

`bmerge` merges $8w$ consecutive places at once, in eight registers, using shuffles. One call is a complete bitonic merge of a block, and not one level of it. This is checked against its trace, for blocks of sixteen, thirty-two and sixty-four places, exactly as section 12 checks the small sorters:

```
Lemma dequiv_bm6 : dequiv 64 bm6 (mlev 6).
```

So this phase is read block size by block size. At size 2^p the code merges as many whole blocks of 2^p as fit in the array. One `minmax_vector` then does what the level at 2^{p-1} still owes after those blocks. Then the size is halved. The four lines after the loop do the same thing at the three smallest sizes, written out. Two definitions describe this. The first says where the code stands after the pass at size 2^p, and the second is the phase itself:

```
Definition bj (n p start : nat) : nat :=
  start + ((n - start) %/ (`2^ p)) * (`2^ p).

Fixpoint bph (n p start : nat) : seq (nat * nat) :=
  if p is p1.+1 then
    flatten [seq pshift (start + m * (`2^ p1.+1)) (mlev p1.+1)
             | m <- iota 0 ((n - start) %/ (`2^ p1.+1))]
    ++ mmv (bj n p1.+1 start) (bj n p1.+1 start + (`2^ p1))
           (n - (`2^ p1) - bj n p1.+1 start)
```

```
    ++ bph n p1 (bj n p1.+1 start)
  else [::].
```

The theorem says that the phase performs exactly the levels it owes. Here `lvsfrom n p start` is every level below 2^p, kept only on the places from `start` on:

```
Theorem bph_levels (n : nat) : forall p start,
  dvdn (`2^ p) start -> start <= n ->
  nequiv n (bph n p start) (lvsfrom n p start).
```

The proof is an induction on p, with one pass per step. It rests on two observations about where the comparisons of a level sit.

The whole blocks are the levels inside them. Cut the array into blocks of 2^p places, starting at a multiple of 2^p. Then a level below 2^p never compares two places in different blocks. Inside one block, the level at distance d **is** the level at d of the merge on that block, moved to where the block sits:

```
Lemma lvin_block (n p d c : nat) :
  d \in dists p -> dvdn (`2^ p) c -> c + `2^ p <= n ->
  lvin n d c (c + `2^ p) = pshift c (level_pairs (`2^ p) d d false).
```

This is an equality of lists, and not a reordering. A level lists its comparisons by increasing low place, so the part inside a block comes out in the order of that block. A place is compared upwards when its d-bit is zero. This condition survives the move, because the block starts at a multiple of $2d$. The second condition, that the comparison stays inside the array, follows from the first one: a place compared upwards is at least d places away from the end of its block. Now take the blocks one after the other, from the front. That gives the first half of a pass:

```
Lemma merges_are_levels (n p start M : nat) :
  dvdn (`2^ p) start -> start + M * (`2^ p) <= n ->
  dequiv n (flatten [seq pshift (start + m * (`2^ p)) (mlev p) | m <- iota 0 M])
           (lvsin n p start (start + M * (`2^ p))).
```

And what is left of a level is one run of comparisons. After the last whole block, less than two blocks of the level at 2^{p-1} remain. So what that level still owes is a single run of consecutive comparisons. That is exactly what one `minmax_vector` does:

```
Lemma lvfrom_tail (n d j : nat) :
  0 < d -> dvdn d.*2 j -> j <= n -> n < j + d.*2 ->
  lvfrom n d j = mm j (j + d) (n - d - j).
```

The rest is a question of order. What a pass does after `bj` stays after `bj`, while the smaller levels are spread over the whole array. The two pieces that must change places share no wire, so one move of a block, by section 6 again, puts the pass in front of the levels below it.

The loop and the phase together fulfil the contract:

```
Corollary bph_mturns (n j k : nat) : 3 <= j -> n <= `2^ (j + 3 * k) ->
  nequiv n (mturns n j k ++ bph n j 0)
           [seq ab <- nstages (half_cleaner_rec false (j + 3 * k)) | ab.2 < n].
```

So the merge of `code/avx2/c/sort_short.c` is the pruned bitonic merge of section 10, at every length. That merge has two parts: the loop, which walks through the array while the distances are large, and the phase, which merges blocks in registers once the distances are small. With a thousand elements, the loop takes two turns, for the levels at 512 down to 16, and the phase does those at 8, 4, 2 and 1.

14 What is proved, and what is not

The five main statements are these.

statement	what it says
`sorted_avx2_prog`	running the model of the AVX2 program returns a sorted array
`sorting_int32_sort_network`	the comparison sequence of the portable code sorts, for every length
`isort_pbsort`	the loop nest of the generic AVX2 sort **is** the bitonic network
`sorting_avx2_short`	the AVX2 program, inside the recursion of section 10, sorts every length
`bph_mturns`	the merge of `sort_short.c`, loop and last phase together, is the pruned bitonic merge

Each of them is closed. Ask Rocq `Print Assumptions` on any of them, and the answer is *closed under the global context*. That means no axiom and no unfinished proof is used.

The statements do not cover the same ground, and it is worth being precise. The portable statement holds for every length. `sorted_avx2_prog` is about arrays whose length is a power of two and at least sixty-four, which is what the vector code is written for. A length that is not a power of two is handled in two ways, and both are proved. One is padding: fill the array with a value above everything else, and drop that filling afterwards. The other is the recursion of section 10, whose blocks are the bits of the length.

One gap remains inside that last statement. From sixty-four elements up, the blocks are sorted by djbsort's own comparisons. Below that size, the C runs a fixed sequence for eight, sixteen and thirty-two elements, and that sequence has not been modelled. A bitonic sort of the same width is used in its place. So at those three sizes the theorem is about a network that the C does not run.

What is **not** proved is the step from the C text to the model of it. In the portable code that gap is written down openly, as the only assumption in the development:

```
Parameter sortc_trace : nat -> seq (nat * nat).
Axiom sortc_faithful : forall n, sortc_trace n = me_pairs n.
```

It says that the comparisons which the C source really performs are the ones the proof works with. To close this gap one needs a semantics for C, which is a separate piece of work. For the moment, the copy is checked by running both versions and comparing their traces, for many sizes, with the small OCaml programs in `code/avx2/ml/` and `code/portable4/ml/`.

The AVX2 code now says the same thing about itself, in `code/avx2/proof/sort_c.v`. It is worth seeing what that assumption is careful **not** to say. It covers lengths that are a power of two and at least sixty-four. In that range the model really is a copy of the code, instruction by instruction. Through padding, it also covers everything that the code sorts by padding. It says nothing about the loops that `code/avx2/c/sort_short.c` runs for a longer array whose length is not a power of two. The model of those loops is the scheme of section 10, which is mathematics and not a copy of the code. That the merge loop performs the pruned merge of that scheme is a proof, not an assumption. Writing it as an axiom would hide the work instead of showing it.

Sections 11 and 13 are that proof, and it is complete. The merge loop performs exactly the levels it should, from the largest distance down to the point where it stops. The phase after it performs exactly the levels below that point, by merging whole blocks in registers with shuffles. Together they are the pruned merge, for every length, and like the statements above they rest on nothing. At the small end, the fixed sequences are covered as well. Sixteen and thirty-two wires are sorted by the code's own comparisons (section 12), and below that the C has no fixed sequence to model.

15 The files

file	lines	what is in it
code/common/nsort.v	804	networks, and what it means to sort
code/common/nbitonic.v	664	the bitonic sorter
code/common/nalgebra.v	1796	the algebra of comparisons: dequiv, nequiv, and their toolkit
code/common/nprog.v	751	programs: Cmp, Vcmp, Vshuf, and pflat
code/common/nprune.v	232	padding, and the merge with comparisons dropped
code/common/nrec.v	454	the recursion for a length that is not a power of two
code/common/nlevel.v	625	the merge one distance at a time, and eight lanes at a time
code/common/nbsl.v	456	the bitonic sorter as a list, and the C's straight lines
code/common/nmloop.v	1369	the merge loop of the AVX2 code, and the phase after it
code/portable4/proof/nbjsort.v	1291	Knuth's merge exchange
code/portable4/proof/int32_knuth.v	602	the portable code is that network
code/avx2/proof/sort_generic.v	592	the bitonic network, and the loop nest
code/avx2/proof/sort_transpose.v	722	the transpose realisation
code/avx2/proof/sort_prog.v	596	djbsort's AVX2 code, as a program
code/avx2/proof/sort_link.v	4426	the bridge for the AVX2 code
code/avx2/proof/sort_short.v	94	that code inside the recursion, at every length
code/avx2/proof/sort_c.v	86	what is still assumed about the C source

The bridge is by far the largest file. That is the honest summary of this note. Proving that a network sorts is textbook work. Proving that a real program performs that network is where the effort goes.

16 The difficult points

The mathematics was not the hard part. The bitonic sorter and Knuth's merge exchange are textbook, and their proofs here are small next to the bridge that connects them to the code. The time went elsewhere. Five points stand out.

Statements that were false. Parts of the bridge were written as a plan: the statements first, the proofs later. Several of those statements were wrong, always in one of two ways. Some claimed an equality between the program's list of comparisons and the network's list, when the two hold the same comparisons in a different order, so that only a reordering can be true. Others forgot to say that the merge fits inside the array, so for a merge wider than the array they spoke about places past the end. Both mistakes appear at once if the statement is tried on a small length before anyone proves it.

Nothing computes. Section 12 says this for networks. A connector carries proofs, so it does not evaluate, and the machine cannot be asked whether a network of sixteen wires sorts. The same is true of the permutations that describe the shuffles. So everything that had to be **checked** rather than proved was written a second time on plain numbers: networks as lists of pairs, permutations as tables. A lemma then ties the two versions together.

A trace names values, not places. The tracer records which values the code compared. While the code only compares, a value and its place are the same thing. After the first shuffle they are not, so a trace taken in the middle of a full run looks like nonsense. Two things work. Model the code in

terms of places, as section 11 does. And trace a single call on a fresh array, as the check of section 13 does.

The parts that do not fill a block. Most of the length of this proof is not about sorting. It is about what the code does with the end of the array, where a block is not full. How many whole blocks of a level a pass covers is arithmetic on a remainder. A length that would be negative gives no comparison, which subtraction on whole numbers already says. And at such an end `minmax_vector` does eight comparisons twice, which is why the statement about it asks for the same function and not the same list. Each point is easy, and none of them could be skipped.

Arithmetic in a prover. Some goals mix a length, that length divided by eight, and powers of two. They are at the limit of what the arithmetic tactics settle quickly, and a goal full of abbreviations pushes them over that limit. The rule that came out of this is simple. Do the arithmetic first, while the goal is still small, and state the lemmas about the shape of the lists separately, so that the two kinds of reasoning never meet in one goal.

17 Summary

The method, in five steps.

1. **Remove every branch from the program.** Then the comparisons it performs are fixed in advance, and the program is a network.
2. **Use zeros and ones.** Sorting is decided by arrays of zeros and ones, and this is what makes induction on networks possible.
3. **Write the program down in a small language,** and read off the comparisons it performs. Rename the places as the shuffles move the values about.
4. **Prove that the order does not matter.** Give the comparisons colours, so that two comparisons of different colours never touch the same place. Sometimes the code does more than reorder. It may do one comparison twice, as `minmax_vector` does at the end of the array. Then ask for less: not the same list, only the same function.
5. **Treat each trick on its own.** Complementing values replaces sorting downwards, and loops replace a recursion. Each one is a small theorem, once it is stated in the right way.

The last step keeps its shape across very different programs. The AVX2 code and the portable code look nothing alike, yet both come down to one sentence: the program compares the same pairs as the network, in an order that costs nothing.